LAB REPORT: STUDENT PERFORMANCE ANALYSIS SYSTEM

Object-Oriented Programming with Java — Laboratory Practical Report

STUDENT NAME	ROLL NUMBER	COURSE / LAB	ACADEMIC YEAR
K. Revanth Reddy	221	Java Programming Lab	2025 – 2026

1. AIM & OBJECTIVE

To design, implement, and test a robust **Student Performance Analysis System** in Java using core Object-Oriented Programming (OOP) concepts. The system accepts student credentials and subject marks, computes total marks, average, highest, lowest, percentage, assigns letter grades, evaluates pass/fail status, cleanses user string inputs, and renders a structured academic report.

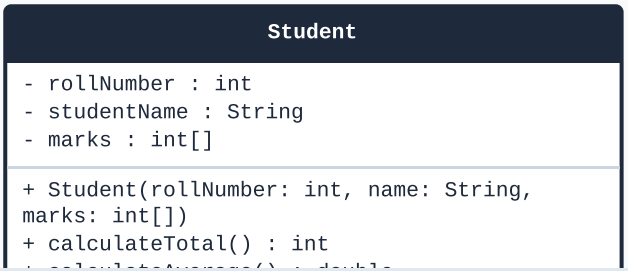
2. PROBLEM STATEMENT

Educational institutions require automated software systems to process student academic results accurately. The application must store marks in integer arrays, calculate statistical metrics using `java.lang.Math` methods (`Math.max()`, `Math.min()`, `Math.round()`), sanitize string inputs with methods such as `trim()` and `toUpperCase()`, evaluate pass criteria (minimum 50% overall), and produce a detailed grade assessment report.

3. STEP-BY-STEP IMPLEMENTATION PROCESS

1. **OOP Class Architecture:** Encapsulate student details (roll number, name, marks array) inside a dedicated class named `Student` using private access modifiers to enforce data integrity.
2. **Input Sanitization:** In the constructor, sanitize student name strings by applying `trim()` to eliminate extra whitespace and `toUpperCase()` for standard display.
3. **Array Processing & Statistical Computation:** Loop through the 5-subject marks array to calculate total and average marks. Utilize `Math.max()` and `Math.min()` inside loops to dynamically track highest and lowest scores.
4. **Percentage Calculation & Rounding:** Compute overall percentage and use `Math.round(percentage * 100.0) / 100.0` to precisely format results to two decimal places.
5. **Grading & Remark Assignment:** Map percentages to letter grades (A+ to F) using multi-way decision logic, assign pass/fail status (≥50% pass threshold), and return a specific performance remark.
6. **Main Driver Execution:** Instantiate the `Student` class in `main()` using the `Scanner` class to gather user inputs and print the consolidated academic output report.

4. UNIFIED MODELING LANGUAGE (UML) CLASS DIAGRAM



5. COMPLETE JAVA SOURCE CODE

```
import java.util.Scanner;

/**
 * Class representing a Student and their academic evaluation methods.
 * Demonstrates Encapsulation, Array operations, Math functions, and String processing.
 */
class Student {
    // Private Data Members (Encapsulation)
    private int rollNumber;
    private String studentName;
    private int[] marks;

    // Constructor to initialize student details
    public Student(int rollNumber, String studentName, int[] marks) {
        this.rollNumber = rollNumber;
        // String Processing: Trim extra spaces and convert to uppercase
        this.studentName = studentName.trim().toUpperCase();
        this.marks = marks;
    }

    // Method to calculate total marks across all subjects
    public int calculateTotal() {
        int total = 0;
        for (int mark : marks) {
            total += mark;
        }
        return total;
    }

    // Method to calculate average marks
    public double calculateAverage() {
        return (double) calculateTotal() / marks.length;
    }

    // Method to find highest marks using Math.max()
    public int findHighest() {
        int highest = marks[0];
        for (int i = 1; i < marks.length; i++) {
            highest = Math.max(highest, marks[i]);
        }
        return highest;
    }

    // Method to find lowest marks using Math.min()
    public int findLowest() {
        int lowest = marks[0];
        for (int i = 1; i < marks.length; i++) {
            lowest = Math.min(lowest, marks[i]);
        }
        return lowest;
    }

    // Method to calculate percentage (assuming maximum 100 marks per subject)
    public double calculatePercentage() {
        int maxPossibleMarks = marks.length * 100;
        return ((double) calculateTotal() / maxPossibleMarks) * 100.0;
    }

    // Method to round percentage to 2 decimal places using Math.round()
    public double getRoundedPercentage() {
        double percentage = calculatePercentage();
        return Math.round(percentage * 100.0) / 100.0;
```

6. EXECUTION OUTPUT CONSOLE

```
=== Student Performance Analysis System ===
Enter Roll Number: 221
Enter Student Name: k. revanth reddy
Enter marks for 5 subjects (out of 100):
Subject 1: 92
Subject 2: 88
Subject 3: 95
Subject 4: 81
Subject 5: 90

================================================
STUDENT PERFORMANCE ANALYSIS REPORT
================================================
Roll Number : 221
Student Name : K. REVANTH REDDY (Length: 16)
------------------------------------------------
Marks Obtained : Sub 1: 92 | Sub 2: 88 | Sub 3: 95 | Sub 4: 81 | Sub 5: 90
------------------------------------------------
Total Marks : 446 / 500
Average Marks : 89.2
Highest Subject Mark : 95
Lowest Subject Mark : 81
Percentage : 89.2%
Grade : A
Result Status : PASSED
Performance Remark : Excellent Job!
================================================
```

7. CORE CONCEPTS & LANGUAGE FEATURES USED

Encapsulation & Abstraction

Private data fields (`rollNumber`, `studentName`, `marks`) restrict direct external access. Data processing logic is exposed safely through public getter methods.

Array Processing & Loops

An integer array (`int[] marks`) stores multiple subject scores. Enhanced `for-each` and standard `for` loops iterate to calculate sums and find mark extremes.

String Operations

The `String` class methods `trim()`, `toUpperCase()`, and `length()` are demonstrated to format user input strings before processing.

Math Package Functions

Built-in `Math.max()` and `Math.min()` compare mark elements dynamically. `Math.round()` formats double precision floats to 2 decimal places.

8. GRADE ALLOCATION MATRIX REFERENCE

Percentage Range	Grade	Status	Performance Remark
90% – 100%	**A+**	PASSED	Outstanding Performance!
80% – 89.99%	**A**	PASSED	Excellent Job!
70% – 79.99%	**B**	PASSED	Good Performance.
60% – 69.99%	**C**	PASSED	Satisfactory, Scope for improvement.
50% – 59.99%	**D**	PASSED	Needs Improvement.